1 线程的问题

1、线程执行完 run 发放自动被销毁了，且任务与线程绑定在了一起，所以当任务多的时候，会频繁的创建和销毁线程，这给我们 CPU 和内存带来了很大的开销。

2、线程一多了，无法实现统一管理。

3、线程很多的时候，CPU 会不断的尽心上下文切换，这也是消耗资源的。

2 线程池的概念及作用

1、他是池化技术的一种应用

2、他实现了线程的重复利用

3、实现了对线程资源的管理控制

3 核心思想

固定的线程数，来消费我们不定量的 task，今天的源码也要基于这个去理解！

我们线程池封装了很多很多的现成的用法

newFixedThreadPool：该方法返回一个固定数量的线程池，线程数不变，当有一个任务提交时，若线程池中空闲，则立即执行，若没有，则会被暂缓在一个任务队列中，等待有空闲的线程去执行。

newSingleThreadExecutor: 创建一个线程的线程池，若空闲则执行，若没有空闲线程则暂缓在任务队列中。

newCachedThreadPool：返回一个可根据实际情况调整线程个数的线程池，不限制最大线程数量，若用空闲的线程则执行任务，若无任务则不创建线程。并且每一个空闲线程会在 60 秒后自动回收

newScheduledThreadPool: 创建一个可以指定线程的数量的线程池，但是这个线程池还带有延迟和周期性执行任务的功能，类似定时器。

newWorkStealingPool:适合使用在很耗时的操作，但是 newWorkStealingPool 不是 ThreadPoolExecutor 的扩展，它是新的线程池类 ForkJoinPool 的扩展，但是都是在统一的一个 Executors 类中实现，由于能够合理的使用 CPU 进行对任务操作（并行操作），所以适合使用在很耗时的任务中

咕泡出品 必属精品

```
    ExecutorService executorService= Executors.newFixedThreadPool(2);
    for (int i = 0; i < 10; i++) {
        executorService.execute(new Thread(() -> {
            System.out.println(Thread.currentThread().getName() + "执行
任务");
        }));
    }
     executorService.shutdown();
}
```

```
ExecutorService executor= Executors.newFixedThreadPool(5);
for (int i = 0; i < 50; i++) {
    executor.execute(
            new Thread(()->{
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                System.out.println("测试"+Thread.currentThread().getNam
e());
            }));
}
executorService.shutdown();
```

其实，除了 newWorkStealingPool，都是对 ThreadPoolExecutor 的一层封装！！并且，建议大家不要用这些封装的，用底层的 ThreadPoolExecutor，这样你就逼着自己去把线程池的一些参数去搞明白！！并且能提供比封装的更多功能！比如监控！！今天我们就去看下 ThreadPoolExecutor 中怎么去实现固定的线程数，来消费我们不定量的 task！！

我们就用最基础的

```
ExecutorService executorService = new ThreadPoolExecutor(2,
        3,
        10,
        TimeUnit.MILLISECONDS
        , new LinkedBlockingQueue<>());
```

3.1 解决自动被销毁

通过 while 死循环，这里出现一个问题，如果没有任务的时候线程也一直在运行，会造成资源浪费，合理的设计是有任务运行，没任务等待。

哈泡出品 必属精品

通过生成者消费者模型解决上面的问题。（这里可以通过 KFC 举例生产者和消费者模型），然后通过代码演示一遍。

3.2 解决线程资源控制

画图解释线程池的工作流程，通过流程解释核心参数。

4 源码分析

4.1 初始化

我们先看下初始化 5 个参数

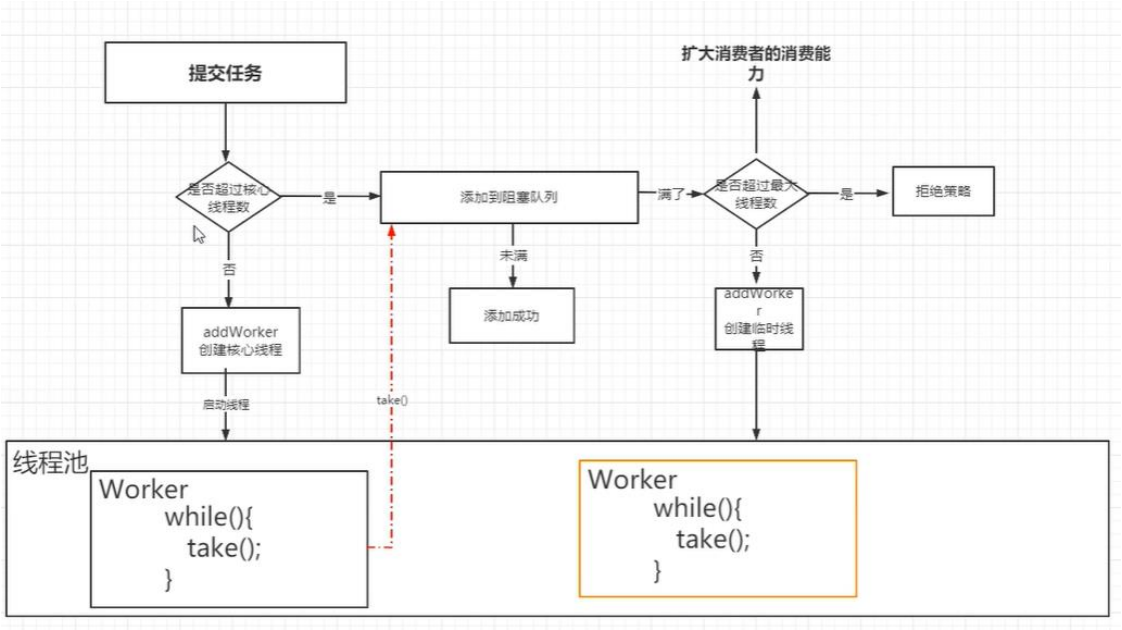
```
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue) {
    this(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue,
         Executors.defaultThreadFactory(), defaultHandler);
}

    public ThreadPoolExecutor(int corePoolSize,  //主线程数
                              int maximumPoolSize,  //最大线程数
                              long keepAliveTime,    //线程存活时间
（除主线程外，其他的线程在没有任务执行的时候需要回收，多久后回收）
                              TimeUnit unit,  //存活时间的时间单位
                              BlockingQueue<Runnable> workQueue,  //阻
塞队列，我们需要执行的task 都在该队列
                              ThreadFactory threadFactory,  //生成threa
d 的工厂
                              RejectedExecutionHandler handler) {  //拒
绝饱和策略，当队列满了并且线程个数达到 maximunPoolSize 后采取的策略
        if (corePoolSize < 0 ||
            maximumPoolSize <= 0 ||
            maximumPoolSize < corePoolSize ||
            keepAliveTime < 0)
            throw new IllegalArgumentException();
        if (workQueue == null || threadFactory == null || handler == nu
ll)
            throw new NullPointerException();
        this.acc = System.getSecurityManager() == null ?
                null :
                AccessController.getContext();
        this.corePoolSize = corePoolSize;
        this.maximumPoolSize = maximumPoolSize;
```

咕泡出品 必属精品

```
        this.workQueue = workQueue;
        this.keepAliveTime = unit.toNanos(keepAliveTime);
        this.threadFactory = threadFactory;
        this.handler = handler;
    }
```

4.2 execute 方法




```
public void execute(Runnable command) {
    if (command == null)  //如果要执行的任务是空的，异常
        throw new NullPointerException();
    /*
     * Proceed in 3 steps:
     *
     * 1. If fewer than corePoolSize threads are running, try to
     * start a new thread with the given command as its first
     * task.  The call to addWorker atomically checks runState and
     * workerCount, and so prevents false alarms that would add
     * threads when it shouldn't, by returning false.
     *
     * 2. If a task can be successfully queued, then we still need
     * to double-check whether we should have added a thread
     * (because existing ones died since last checking) or that
     * the pool shut down since entry into this method. So we
     * recheck state and if necessary roll back the enqueuing if
     * stopped, or start a new thread if there are none.
     *
     * 3. If we cannot queue task, then we try to add a new
```

```
     * thread.  If it fails, we know we are shut down or saturated
     * and so reject the task.
     */
    int c = ctl.get();//111000...000
    //高三位代表线程池的状态，低29位代表线程池中的线程数量
    //如果线程数小于主线程数，添加线程
    if (workerCountOf(c) < corePoolSize) {
        if (addWorker(command, true))
            return;
        c = ctl.get();
    }
    //如果超过主线程数，将任务添加至workqueue 阻塞队列
    if (isRunning(c) && workQueue.offer(command)) {
        int recheck = ctl.get();
        //再判断一次运行状态，如果线程池不处于running状态，则把刚加进队列的
任务移除，如果移除成功则往下走进行拒绝
        if (! isRunning(recheck) && remove(command))
            reject(command);
        //接着上一个条件，如果移除失败则判断是否有工作线程，如果当前线程池线
程空，则添加一个线程
        else if (workerCountOf(recheck) == 0)
            addWorker(null, false);
    }
    //如果超过主线程数且添加阻塞队列失败，则增加非核心线程，如果添加非核心线
程也失败，则拒绝
    else if (!addWorker(command, false))
        reject(command);
}


private final AtomicInteger ctl = new AtomicInteger(ctlOf(RUNNING, 0));
//111000...000
private static final int COUNT_BITS = Integer.SIZE - 3;//29
private static final int CAPACITY   = (1 << COUNT_BITS) - 1;//00011111
11111111 11111111 11111111
//00000000 00000000 00000000 00000001  << 29 =
//00100000 00000000 00000000 00000000 -1 =
//00011111 11111111 11111111 11111111

// runState is stored in the high-order bits
private static final int RUNNING    = -1 << COUNT_BITS; //11100000 ...
000
//-1 原码: 10000000 00000000 00000000 00000001
//-1 反码: 11111111 11111111 11111111 11111110
//-1 补码: 11111111 11111111 11111111 11111111 <<29=
//        11100000 0000000 00000000 00000000

private static final int SHUTDOWN   =  0 << COUNT_BITS;//00000000 ... 0
```

曹磊出品 必属精品

```
00
private static final int STOP       =  1 << COUNT_BITS;//001 0000 ... 0
00
private static final int TIDYING    =  2 << COUNT_BITS;//010 0000 ... 0
00
private static final int TERMINATED =  3 << COUNT_BITS;//011 0000 ... 0
00
```

1、RUNNING

(1) 状态说明：线程池处在 RUNNING 状态时，能够接收新任务，以及对已添加的任务进行处理。

(02) 状态切换：线程池的初始化状态是 RUNNING。换句话说，线程池被一旦被创建，就处于 RUNNING 状态，并且线程池中的任务数为 0！

```
private final AtomicInteger ctl = new AtomicInteger(ctlOf(RUNNING, 0));
```

2、 SHUTDOWN

(1) 状态说明：线程池处在 SHUTDOWN 状态时，不接收新任务，但能处理已添加的任务。

(2) 状态切换：调用线程池的 shutdown()接口时，线程池由 RUNNING -> SHUTDOWN。

3、STOP

(1) 状态说明：线程池处在 STOP 状态时，不接收新任务，不处理已添加的任务，并且会中断正在处理的任务。

(2) 状态切换：调用线程池的 shutdownNow()接口时，线程池由(RUNNING or SHUTDOWN) -> STOP。

4、TIDYING

(1) 状态说明：当所有的任务已终止，ctl 记录的"任务数量"为 0，线程池会变为 TIDYING 状态。当线程池变为 TIDYING 状态时，会执行钩子函数 terminated()。terminated()在 ThreadPoolExecutor 类中是空的，若用户想在线程池变为 TIDYING 时，进行相应的处理；可以通过重载 terminated()函数来实现。

(2) 状态切换：当线程池在 SHUTDOWN 状态下，阻塞队列为空并且线程池中执行的任务也为空时，就会由 SHUTDOWN -> TIDYING。

当线程池在 STOP 状态下，线程池中执行的任务为空时，就会由 STOP -> TIDYING。

5、 TERMINATED

(1) 状态说明：线程池彻底终止，就变成 TERMINATED 状态。

(2) 状态切换：线程池处在 TIDYING 状态时，执行完 terminated()之后，就会由 TIDYING -> TERMINATED。

```
private static int runStateOf(int c)     { return c & ~CAPACITY; }
private static int workerCountOf(int c)  { return c & CAPACITY; }//CAPACITY:000111...111
private static int ctlOf(int rs, int wc) { return rs | wc; }
```

4.3 addWorker 方法

```
private boolean addWorker(Runnable firstTask, boolean core) {
    retry:  //goto 语句  叫demo
    //自旋检查线程池的状态。阻塞队列是否为空等判断
    for (;;) {
        int c = ctl.get();
        int rs = runStateOf(c);
```

咕泡出品必属精品

```
        // Check if queue empty only if necessary.
        if (rs >= SHUTDOWN &&
            ! (rs == SHUTDOWN &&
               firstTask == null &&
               ! workQueue.isEmpty()))//如果线程池的运行状态是running 的话直接跳过该条件语句往下走，如果是>=SHUTDOWN 的话就往后判断（为什么不直接返回false 不让他创建worker 呢，因为在shutdown 状态是可以创建线程去处理阻塞队列里的任务的）
            //此时因为rs>=SHTDOWN 了，所以会先判断是否等于SHUTDOWN，如果不等于就直接返回false 不让创建worker，如果等于的话接着往下判断
            //如果当前任务不为空直接返回false 不让创建worker，（这里为什么当前任务为空就直接不让创建worker 呢，就是因为shutdown 状态不能再接收新任务。
            //如果当前任务为空则判断阻塞队列是否为空，如果为空则返回false，不让创建worker，如果不为空就不走这个条件，接着往下走
            return false;

        //自旋
        for (;;) {
            int wc = workerCountOf(c);
            //如果现有线程数大于最大值，或者大于等于最大线程数（主线程数）
            if (wc >= CAPACITY ||
                wc >= (core ? corePoolSize : maximumPoolSize))
                return false;
            //cas 添加线程
            if (compareAndIncrementWorkerCount(c))
                break retry;
            c = ctl.get();  // Re-read ctl
            //如果失败了，继续外层循环判断
            if (runStateOf(c) != rs)
                continue retry;
            // else CAS failed due to workerCount change; retry inner loop
        }
    }

    boolean workerStarted = false;
    boolean workerAdded = false;
    Worker w = null;
    try {
        //开启一个线程，Worker 实现了runnable 接口
        w = new Worker(firstTask);
        final Thread t = w.thread;
        if (t != null) {
            final ReentrantLock mainLock = this.mainLock;
            mainLock.lock();
            try {
                // Recheck while holding lock.
```

咕泡出品 必属精品

```
                // Back out on ThreadFactory failure or if
                // shut down before lock acquired.
                int rs = runStateOf(ctl.get());

                if (rs < SHUTDOWN ||
                    (rs == SHUTDOWN && firstTask == null)) {
                    if (t.isAlive()) // precheck that t is startable
                        throw new IllegalThreadStateException();
                    //添加至wokers
                    workers.add(w);
                    int s = workers.size();
                    if (s > largestPoolSize)
                        largestPoolSize = s;
                    workerAdded = true;
                }
            } finally {
                mainLock.unlock();
            }
            //添加成功
            if (workerAdded) {
                t.start();   //启动线程，会调用我们线程的run 接口，也就是我
们worker 的run
                workerStarted = true;
            }
        }
    } finally {
        if (! workerStarted)
            addWorkerFailed(w);
    }
    return workerStarted;
}
```

4.4 goto 语句 demo

```
    retry:
        for (int i = 0; i < 3; i++) {
            for (int j = 3; j < 10; j++) {
//                if (j == 4) {
//                    break retry;  //跳出外面循环
//                }
                if (j == 7) {
                    continue retry;  //继续外面循环
                }
                System.out.println(i+":"+j);
            }

        }
```

咕泡出品 必属精品

```
Worker(Runnable firstTask) {
    setState(-1); // inhibit interrupts until runWorker   禁止中断，直到runWorker
    this.firstTask = firstTask;
    this.thread = getThreadFactory().newThread(this);
}
```

4.5 worker run 方法

```
final void runWorker(Worker w) {
    Thread wt = Thread.currentThread();
    Runnable task = w.firstTask;
    w.firstTask = null;
    w.unlock(); // allow interrupts
    boolean completedAbruptly = true;
    try {        //只要一直能获取到task，就一直会执行，不会关闭，所以线程也不会销毁，线程销毁只有当task 为null
        while (task != null || (task = getTask()) != null) {

            w.lock();
            // If pool is stopping, ensure thread is interrupted;

            // if not, ensure thread is not interrupted.  This

            // requires a recheck in second case to deal with

            // shutdownNow race while clearing interrupt
            if ((runStateAtLeast(ctl.get(), STOP) ||
                 (Thread.interrupted() &&
                  runStateAtLeast(ctl.get(), STOP))) &&
                !wt.isInterrupted())
                wt.interrupt();
            try {
                //调用线程方法之前执行
                beforeExecute(wt, task);
                Throwable thrown = null;
                try {
                    //调用task 的run 方法
                    task.run();
                } catch (RuntimeException x) {
                    thrown = x; throw x;
                } catch (Error x) {
                    thrown = x; throw x;
                } catch (Throwable x) {
```

咕泡出品 必属精品

```
                    thrown = x; throw new Error(x);
                } finally {
                    //调用线程方法之后执行
                    afterExecute(task, thrown);
                }
            } finally {
                task = null;
                w.completedTasks++;
                w.unlock();
            }
        }
        completedAbruptly = false;
    } finally {
        processWorkerExit(w, completedAbruptly);
    }
}
```

4.6 getTask()方法

```
private Runnable getTask() {
    boolean timedOut = false; // Did the last poll() time out?    //自旋获取

    for (;;) {
        int c = ctl.get();
        int rs = runStateOf(c);

        // Check if queue empty only if necessary. 必要时检查空，状态是否停止或者shutdown
        if (rs >= SHUTDOWN && (rs >= STOP || workQueue.isEmpty())) {

            decrementWorkerCount();
            return null;
        }

        //获取线程数量
        int wc = workerCountOf(c);

        // Are workers subject to culling?
        //线程数大于主线程数时，或者allowCoreThreadTimeOut 参数为true  allowCoreThreadTimeOut 默认为false
        boolean timed = allowCoreThreadTimeOut || wc > corePoolSize;

        //超过最大线程，或者timed 为true ,&& wc 大于1 个，并且任务队列为空的时候
        if ((wc > maximumPoolSize || (timed && timedOut))
            && (wc > 1 || workQueue.isEmpty())) {
```

咕泡出品 必属精品

```
            //线程数-1，并且返回null，该线程结束
            if (compareAndDecrementWorkerCount(c))
                return null;
            continue;
        }

        try {
            //如果time 是true，超过时间不阻塞，不然一直阻塞，不回收

            Runnable r = timed ?
                workQueue.poll(keepAliveTime, TimeUnit.NANOSECONDS) :
            //移除并返回队列头部的元素，如果为空，超过时间返回null

            workQueue.take();
            //移除并返回队列头部的元素，如果为空,一直阻塞
            if (r != null)
                return r;
            timedOut = true;
        } catch (InterruptedException retry) {
            timedOut = false;
        }
    }
}
```

5 最大线程数什么场景才会用到？

task 线程添加不进去，满了

举例设置阻塞队列的大小

```
ExecutorService executorService = new ThreadPoolExecutor(2,3,10,TimeUni
t.MILLISECONDS,
                                                        new LinkedBloc
kingQueue<>(3),
                                                        new ExecJavaTe
mplate());
for (int i = 0; i < 10; i++) {
    executorService.execute(new Thread(() -> {
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(Thread.currentThread().getName() + "执行任务
");
```

哈尼出品 必属精品

```
    }));
}
```

拒绝策略

```
public class ExecJavaTemplate implements RejectedExecutionHandler {
    @Override
    public void rejectedExecution(Runnable r, ThreadPoolExecutor execut
or) {
        System.out.println(r.toString());
    }
}
```

6 线程的回收

getTask()返回 null,线程结束

7 线程池关闭

```
public void shutdown() {
    final ReentrantLock mainLock = this.mainLock;
    mainLock.lock();
    try {
        // 权限校验，安全策略相关判断
        checkShutdownAccess();
        // 设置SHUTDOWN 状态
        advanceRunState(SHUTDOWN);
        // 中断所有的空闲的工作线程
        interruptIdleWorkers();
        onShutdown();
        // hook for ScheduledThreadPoolExecutor
    } finally {
        mainLock.unlock();
    }
    tryTerminate();
}
```

叶龙出品 必属精品

注意，shutdown 不会终止正在运行的线程，只能通过线程的状态去自旋中断，然后结束线程

咕泡出品 必属精品